

(2) Plotting

<https://www.kaggle.com/code/themlphdstudent/cheat-sheet-seaborn-charts?scriptVersionId=176781102&cellId=1>

(seaborn cheatsheet - you surely will click this)

Multi-Plot / Panel Plots

Create a grid of subplots

```
fig, axes = plt.subplots(  
    nrows, ncols,  
    figsize=(W, H),  
    sharex=False,  
    sharey=False  
)
```

- **fig** → Figure (entire canvas / page)
- **axes** → array of Axes objects (each Axes = one subplot)
- **axes** is a **NumPy array**
- Shape: (W, H)

```
fig, axes = plt.subplots(2, 2)    # 2x2 panel  
fig, axes = plt.subplots(4, 4)    # 4x4 panel
```

Each **Axes** object is a **full plotting area**.

You always plot on an Axes:

```
ax.plot(...)  
ax.hist(...)  
ax.scatter(...)  
ax.set_title(...)
```

Flatten axes for easy looping

```
axes = axes.flatten()  
  
for ax in axes:  
    ax.plot(...)
```

Example

```

fig, axes = plt.subplots(
    4, 4,
    figsize=(14, 14),
    sharex=True,
    sharey=True
)

axes = axes.flatten()

for ax, genre in zip(axes, genres):
    data = movies.loc[movies["Main_Genre"] == genre, "length"]
    ax.hist(data, bins=20, range=[0, 200])
    ax.set_title(genre, fontsize=9)

fig.suptitle("Distribution of Movie Lengths by Genre", fontsize=16,
y=0.98)
fig.supxlabel("Movie Length (minutes)")
fig.supylabel("Number of Movies")

fig.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

Heatmaps with `pandas.crosstab` + Seaborn

What is a heatmap?

A **heatmap** visualizes a 2D table of values using color intensity.

- Rows → categories (y-axis)
- Columns → categories (x-axis)
- Cell color → magnitude of the value
- Optional annotations show the exact numbers

```
pd.crosstab(row_var, col_var, values=None, aggfunc=None)
```

`crosstab` creates a **contingency table** (matrix) that summarizes the relationship between two categorical variables.

By default

- Counts the number of occurrences for each (row, column) combination
With `values + aggfunc`
- Aggregates numeric data instead of counting

Heatmap 1 — Count of movies by genre and studio

```
df2 = pd.crosstab(movies["Main_Genre"], movies["studio"])
sns.heatmap(df2, annot=True, vmin=0, vmax=20)

# Movie Rank and Count
df4 = pd.crosstab(movies["Main_Genre"], movies["rank_in_year"])
sns.heatmap(df4,annot=True,vmin = 0, vmax=20)
plt.show()
```

Heatmap 2 — Mean worldwide gross by genre combinations

```
df3 = pd.crosstab(
    movies["Main_Genre"],
    movies["Genre_2"],
    values=movies["worldwide_gross"],
    aggfunc="mean"
)
sns.heatmap(df3, annot=False)
```

```
pd.crosstab(
    rows,
    cols,
    values=NUMERIC_SERIES,
    aggfunc="mean" | "sum" | "median",
    margins=True # adds totals
)

sns.heatmap(
    data,
    annot=True,
    cmap="viridis",
    vmin=0,
    vmax=20
)
```

Line plot

```
plt.plot(x, y) plt.show()
```

Scatter plot

```
plt.scatter(x, y) plt.show()
s=40
```

c=values
cmap="viridis"

Bar plot

```
plt.bar(categories, values) plt.show()  
width=0.8
```

Horizontal bar plot

```
plt.barh(categories, values) plt.show()
```

Histogram

```
plt.hist(data, bins=20) plt.show()  
bins=20  
range=(0, 100)  
density=True
```

Box plot

```
plt.boxplot(data) plt.show()
```

Violin plot

```
plt.violinplot(data) plt.show()
```

Pie chart

```
plt.pie(values, labels=labels, autopct="%1.1f%%") plt.show()
```

Area plot

```
plt.fill_between(x, y) plt.show()
```

Step plot

```
plt.step(x, y) plt.show()
```

Error bar plot

```
plt.errorbar(x, y, yerr=errors, fmt="o") plt.show()
```

Multiple lines on one plot

```
plt.plot(x, y1) plt.plot(x, y2) plt.show()
```

```
# Line / Marker styling (most plots)
color="C0"
linewidth=2
linestyle="--"          # "-", "--", "-.", ":"
marker="o"              # "o", "s", "^", "x", ".", "+"
markersize=6
markerfacecolor="white"
markeredgecolor="black"
alpha=0.8
label="Series name"

# Text And Annotations
plt.title("Plot Title", fontsize=14)
plt.xlabel("X label", fontsize=12)
plt.ylabel("Y label", fontsize=12)
plt.text(x, y, "text", fontsize=10)
plt.annotate(
    "label",
    xy=(x, y),
    xytext=(x2, y2),
    arrowprops=dict(arrowstyle="->")
)

# Axis limits & scales
plt.xlim(xmin, xmax)
plt.ylim(ymin, ymax)
plt.xscale("linear")    # "log"
plt.yscale("linear")    # "log"
```

```
# Ticks
plt.xticks([0, 10, 20])
plt.yticks([0, 50, 100])
plt.xticks([0, 1, 2], ["A", "B", "C"])
#### Special example
positions = plt.xticks()[0]
plt.xticks(positions, ["A", "B", "C"])
####
plt.xticks(rotation=45)

# Grid
plt.grid(True)
plt.grid(axis="y", linestyle="--", alpha=0.6)

# Legend
plt.legend()
plt.legend(
    loc="best",          # "upper right", "lower left", etc.
    fontsize=10,
    frameon=False
)

# Figure & Layout
plt.figure(figsize=(8, 6))
plt.tight_layout()

# Save Figure
plt.savefig(
    "plot.png",
    dpi=300,
    bbox_inches="tight"
)
```

